DonorConnect - Cloud Based Donation & Fundraising Management Platform

Project Overview

DonorConnect is a cloud-based donation and fundraising management platform designed to help organizations manage donors, fundraising campaigns, donations, and reports through a centralized web application.

The platform provides administrators with an easy-to-use dashboard for managing donor information, tracking fundraising campaigns, monitoring donation statistics, and generating reports.

This project demonstrates deployment of a Flask-based web application on AWS EC2 integrated with AWS RDS MySQL for cloud database management.

Objectives

The primary objectives of the project are:

- Develop a web-based donation management system.
- Centralize donor and campaign information.
- Provide real-time fundraising statistics.
- Demonstrate cloud deployment using AWS services.
- Implement CRUD operations using a relational database.
- Enable secure administrator login and session management.

Technology Stack

Frontend

- HTML5
- CSS3
- Bootstrap
- Jinja2 Templates

Backend

- Python
- Flask Framework

Database

- MySQL
- AWS RDS MySQL

Cloud Services

- AWS EC2
- AWS RDS

Development Tools

- VS Code
- Git
- DB Browser
- MySQL Client

System Architecture

User Browser ↓ AWS EC2 Instance ↓ Flask Application ↓ MySQL Connector ↓ AWS RDS MySQL Database

Architecture Components

Client Layer

The user interacts with the application using a web browser.

Application Layer

Flask processes requests, performs business logic, and communicates with the database.

Database Layer

AWS RDS MySQL stores all donor, campaign, and administrator records.

Cloud Infrastructure

AWS EC2 hosts the Flask application and serves web pages to users.

AWS Services Used

AWS EC2

Purpose:

- Host the Flask application.
- Execute backend business logic.
- Serve web requests.

Configuration:

- Ubuntu Server

- Public IP Access
- Security Group Configuration

AWS RDS

Purpose:

- Store application data.
- Manage donor and campaign records.
- Provide managed MySQL database service.

Configuration:

- MySQL Engine
- Public Access Enabled
- Database Name: donorconnect

Features Implemented

Authentication Module

Admin Login

Features:

- Secure Login Form
- Session Management
- Credential Verification

Route:

/login

Methods:

POST

Dashboard Module

Features:

- Total Donors Count
- Total Donations Amount
- Total Campaign Count
- Recent Donation Activity

Route:

/dashboard

Donor Management Module

Features:

- Add Donor
- View Donors
- Search Donors
- Edit Donor
- Delete Donor

Routes:

/donors /add_donor /edit_donor/ /update_donor/ /delete_donor/

Campaign Management Module

Features:

- Create Campaign
- View Campaigns
- Update Campaigns
- Delete Campaigns
- Campaign Progress Tracking

Routes:

/campaigns /add_campaign /edit_campaign/ /update_campaign/ /delete_campaign/

Reports Module

Features:

- Total Donors
- Total Donations
- Average Donation
- Campaign Statistics

Route:

/reports

Database Design

Database Name

donorconnect

Table: admins

Column	Type
id	INT
admin_id	VARCHAR
password	VARCHAR

Purpose: Stores administrator login credentials.

Table: donors

Column	Type
id	INT
name	VARCHAR
email	VARCHAR
phone	VARCHAR
city	VARCHAR
amount_donated	DECIMAL

Purpose: Stores donor information and donation amounts.

Table: campaigns

Column	Type
id	INT
title	VARCHAR
description	TEXT
target_amount	DECIMAL
raised_amount	DECIMAL
start_date	DATE
end_date	DATE

Purpose: Stores campaign information and fundraising goals.

Project Workflow

Step 1

Administrator opens the application.

↓

Step 2

Administrator logs in using credentials.

↓

Step 3

Flask validates credentials using AWS RDS.

↓

Step 4

Dashboard displays analytics and donation statistics.

↓

Step 5

Administrator manages donors and campaigns.

↓

Step 6

Database updates are stored in AWS RDS.

↓

Step 7

Reports are generated dynamically.

Security Features

- Session-Based Authentication
- SQL Parameterized Queries
- Server-Side Validation
- Protected Routes
- Database Isolation via RDS

Folder Structure

```
donorconnect/
├── app.py
├── db.py
├── templates/
│ ├── login.html
│ ├── dashboard.html
│ ├── donors.html
│ ├── edit_donor.html
│ ├── campaigns.html
│ ├── edit_campaign.html
│ ├── reports.html
│ └── architecture.html
│
├── static/
├── requirements.txt
└── README.md
```

Deployment Process

EC2 Setup

1. Launch Ubuntu EC2 Instance.
2. Configure Security Group.
3. Connect using SSH.
4. Install Python and dependencies.

RDS Setup

1. Create MySQL RDS instance.
2. Configure database access.
3. Create donorconnect database.
4. Create required tables.

Application Deployment

1. Upload project files to EC2.
2. Configure database connection.
3. Install dependencies.
4. Run Flask application.

Command:

python3 app.py

Application URL:

http://:5003

Testing Performed

Test Case	Status
Admin Login	Passed
Dashboard Load	Passed
Add Donor	Passed
Edit Donor	Passed
Delete Donor	Passed
Search Donor	Passed
Add Campaign	Passed
Update Campaign	Passed
Delete Campaign	Passed
Reports Generation	Passed
AWS RDS Connection	Passed
AWS EC2 Deployment	Passed

Future Enhancements

- Password Hashing
- Role-Based Access Control
- Donor Payment Gateway Integration
- Email Notifications
- PDF Report Generation
- AWS S3 File Storage

- CloudWatch Monitoring
- Docker Containerization
- CI/CD Deployment Pipeline

Learning Outcomes

Through this project, the following concepts were learned:

- Flask Web Development
- AWS EC2 Deployment
- AWS RDS Integration
- MySQL Database Management
- CRUD Operations
- Session Management
- Cloud Infrastructure Basics
- Web Application Architecture
- Client-Server Communication

Conclusion

DonorConnect successfully demonstrates a cloud-hosted donation and fundraising management platform using Flask, AWS EC2, and AWS RDS. The system provides secure administrator access, donor management, campaign tracking, reporting capabilities, and cloud database integration, making it a scalable foundation for real-world fundraising applications.